Midterms Pt 1 Report: Sandboxed, Role-Based MCP File-Server Connector

Author: Ezekiel Jairus Solitario, BSCS 3A | **Course:** CS 3A / CS 3B Elective, Week 6 (Model Context Protocol)

Repository: https://github.com/zekeyell/Agent-handoff-mcp | **Public endpoint:** https://neatness-daredevil-ion.ngrok-free.dev (POST /mcp/dev and /mcp/qa, bearer token required)

1. Overview

I built an MCP file-server connector that lets several AI agents share one sandboxed workspace. Each agent gets only the tools its role needs: `/mcp/dev` exposes all 8 tools and `/mcp/qa` exposes 4 (`read_file`, `search_code`, `git_status`, `git_commit`). Context moves between sessions through four files in the workspace (PROJECT_STATE, PLAN_LOG, CHECKPOINTS, DECISIONS), so a fresh chat can resume work without being briefed.

I proved it with a three-session Flutter/Firebase Sprint run from VS Code in Agent mode. Every session was a new chat with only the connector's tools enabled.

- **Session 1 (frontend, dev role):** scaffolded a Material 3 Flutter app called Sprint Board behind a `TaskRepository` interface.
- **Session 2 (backend, dev role):** resumed from the handoff files and added Firebase (anonymous auth and Firestore) without changing the UI code.
- **Session 3 (QA, read-only role):** audited the result and tried seven sandbox escapes, all rejected by the server.

The workspace git history shows each step, with `[dev]` and `[qa]` commit prefixes.

2. Stack and justification

Layer	Choice	Alternative considered	Why
Runtime and SDK	Node.js with the official TypeScript MCP SDK	Python with FastMCP	One language for server, tools and tests; the reference SDK tracks the spec closely
Transport	Streamable HTTP, stateless	stdio	The server must be reachable by remote clients and survive restarts; stdio only serves the host that spawned it
Validation	Zod on every tool input	Hand-written JSON Schema	One schema both validates input and is the tool listing the agent reads
Exposure	ngrok with a static domain	Cloudflare Tunnel	Free static domain, one command, and a local request inspector that doubles as evidence
Process manager	pm2 running the server and the tunnel	systemd or Docker	Works on Windows, where I develop, and restarts both processes automatically
Version control	git inside the workspace	none	Every checkpoint is a commit, so the history is the audit trail
Testing	Node test runner, MCP Inspector, a real client (VS Code)	Scripted JSON-RPC only	Unit tests for the sandbox, integration tests against the real server, then a real client

Why stateless. Each POST builds its own server and transport object. A pm2 restart or a tunnel reconnect therefore breaks nothing, and continuity comes from the handoff files rather than server memory. The server binds to 127.0.0.1 only, so the ngrok tunnel is the

single public entrance. Configuration is split into server (`src/index.ts`), security (`src/sandbox.ts`) and tools (`src/tools.ts`), and every secret lives in environment variables.

3. Tool design choices

The 8 tools are `list_files` , `read_file` , `write_file` , `str_replace` , `delete_file` , `search_code` , `git_status` and `git_commit` . Each does one thing, and each description states what it does, when to use it and how it fails, because the description is the prompt the agent reads.

- **`str_replace` requires a unique match.** It fails if the text is missing or appears more than once, and reports the line numbers, which prevents silent wrong edits and saves tokens compared with rewriting files.
- **`read_file` truncates at 1000 lines** and appends a system note telling the agent to use `start_line` and `end_line` . Ranges are capped at 1000 lines per call.
- **`search_code` uses `git grep` through `execFile`** with an argument array, so there is no shell and no injection. It lets agents locate code instead of reading whole files.
- **`delete_file` is gated.** It needs an explicit `confirm: true` , refuses directories, and can never delete the four protocol files, with or without `.md` and in any letter case.
- **`git_commit` records who acted.** The message gets a `[dev]` or `[qa]` prefix and the commit author is `mcp-dev` or `mcp-qa` . An `allow_empty` option lets QA record an audit marker commit.
- **Errors are built for recovery.** Every failure returns `isError: true` with a message that says what to do next. Messages never contain stack traces, and the host path is replaced with `<workspace>` .
- **Writes are atomic.** `write_file` and `str_replace` write to a temp file and rename it, so a crash cannot leave a half-written file.

4. Agent and tool management

Role	Endpoint	Tools	Purpose
dev	`/mcp/dev`	all 8	Frontend and backend development sessions

Role	Endpoint	Tools	Purpose
qa	`/mcp/qa`	`read_file`, `search_code`, `git_status`, `git_commit`	Read-only audit; can record the verdict as a commit

Each endpoint registers only its own tools, so a tool a role does not have does not exist for it. The test suite checks that QA cannot call `write_file`, `str_replace`, `delete_file` or `list_files`, and the log records such attempts.

Handoff protocol. The server creates the four protocol files on first start and sends the rules to every agent in its server instructions: read PROJECT_STATE and the end of the logs first, write the plan to PLAN_LOG before acting, commit and record the hash in CHECKPOINTS after each milestone, record choices in DECISIONS, and finish by overwriting PROJECT_STATE with the exact next action.

These excerpts from the handoff files show the second session resuming from the first.

PROJECT_STATE.md (written at the end of Session 1, read at the start of Session 2):

```
- Next action (be exact): Implement and configure a Firebase-backed
TaskRepository, then run Flutter dependency resolution, analysis, and tests
in a Flutter-enabled environment.
```

PLAN_LOG.md:

```
S1 [dev] DONE: Scaffolded the app, repository, board, dialog, and task card;
source-level checks passed.
S2 [dev] PLAN: Add Firebase Core/Auth/Firestore dependencies, initialize
Firebase and anonymous auth before app startup, swap repository injection to
a Firestore TaskRepository, add clearly marked placeholder options,
authenticated task-field rules, and manual setup docs; verify source-level
contracts because Flutter/Firebase CLIs are unavailable; commit
implementation milestone, record decisions and checkpoint, overwrite handoff
state with next action "QA audit", then commit handoff files.
S2 [dev] DONE: Implemented Firebase startup, Firestore task persistence,
placeholder options, authenticated field-validation rules, and manual setup
instructions; source-level checks passed. Flutter dependency resolution,
analysis, and runtime tests remain for QA in a Flutter-enabled environment.
Implementation commit: cfa71dc.
```

CHECKPOINTS.md:

```
S1 2354965 Flutter app scaffold with in-memory board, add/move/delete
interactions.
S2 cfa71dc Firebase Core/Auth/Firestore integration, anonymous sign-in,
repository, placeholder options, rules, and setup guide.
```

The prompt for Session 2 described the goal only. The file list, the commit hash and the next action it needed came from PROJECT_STATE, and its plan follows the next action Session 1 left. Session 3 then checked those same two recorded hashes, `2354965` and `cfa71dc`, against the git history.

Known gap. QA has no write tool, so it cannot append to the handoff files. Its verdict lives in the audit commit (`be44eec`) and in its transcript.

5. Security and tradeoffs

Authentication. The server refuses to start unless `MCP_AUTH_TOKEN` is set (16 characters or more). Every request on every route needs `Authorization: Bearer <token>`, compared in constant time, and anything else gets `401 Unauthorized`. Failures are logged to `logs/auth-failures.jsonl` without the token. I confirmed the `401` on localhost and through the public ngrok URL.

Sandbox. `src/sandbox.ts` runs before any file operation and rejects, in order:

1. null bytes, empty paths and paths over 1024 characters;
2. absolute, UNC and drive-letter paths, and any path containing `:`;
3. any `..` segment and dot-only segments (a Windows quirk);
4. `.git`, including case and trailing-dot variants;
5. symlinks and junctions whose real target is outside the root, and broken symlinks (which blocks writing through a dangling link).

Testing. `npm test` runs 21 tests, all passing: 8 sandbox unit tests and 13 integration tests that start the real server. The integration tests cover the boot failure without a token, `401` on every route without a token, the role tool lists, sandbox rejection through the tools, the gated delete, truncation, unique replacement, git attribution and the log format. In Session 3 the QA agent also issued seven escape attempts through a real client, and the server rejected all seven (`..`, `../../etc/passwd`, an absolute path, a drive letter, backslash traversal, `lib/../../x` and `.git/config`).

Threats considered: path traversal, symlink escape, shell injection, token theft, accidental destructive calls and an agent overstepping its role.

6. Results of the three sessions

Session	Role	Task	Tool calls	Errors	Avg ms	Output returned	Commits
1	dev (frontend)	Flutter UI scaffold	33	0	108	3,786 chars	2354965, da509c2
2	dev (backend)	Firebase integration	34	0	84	7,796 chars	cfa71dc, ae23b24
3	qa	Audit and sandbox probes	29	7 (deliberate)	62	18,463 chars	be44eec

The seven Session 3 errors are the sandbox probes, not failed work. The workspace history is: `118eba1` (seed), `2354965`, `da509c2`, `cfa71dc`, `ae23b24`, `be44eec`. Session 3 ended with the verdict PASS, 0 findings.

7. Efficiency analysis

- **No failed calls in the development sessions.** Sessions 1 and 2 made 67 calls with 0 errors, and the agents never retried a call without new information.
- **Targeted edits.** Session 1 used 9 `search_code` calls and 3 `str_replace` edits; Session 2 used 8 `search_code` calls and 5 `str_replace` edits. Neither session rewrote files it only needed to change.
- **Small outputs.** No call hit the 1000-line truncation, and Session 1 returned about 3.8 thousand characters in total. Session 2 read more (3,922 characters over 8 `read_file` calls) because it first had to read the handoff files and the existing code.
- **QA read the most.** Session 3 returned 18,463 characters because it read 11 files for the audit; it still finished in 29 calls over about 34 seconds.
- **Plan before action.** Both development agents wrote their plan to PLAN_LOG before touching code, as the protocol requires.

The raw numbers are in `submission/session1-summary.md`, `session2-summary.md` and `session3-summary.md`, produced by `npm run logs:summary` from `logs/tool-calls.jsonl`.

8. Problems and lessons learned

1. **Rejected calls were missing from the log.** My integration tests showed that the SDK rejects bad input and calls to tools a role does not have before my handler runs, so those attempts were not logged. I added a hook that logs them, marked `rejected_by: protocol`, which also records role-gating attempts.
2. **pm2 could not launch the Store version of ngrok on Windows.** It looked for a file named `ngrok` in the project folder. I patched `ecosystem.config.cjs` to start it through `cmd`. Closing the console window stops the tunnel, and pm2 restarts it, which is why that process showed several restarts until I reset the counter.
3. **DNS failure.** My router returned only IPv6 addresses for the ngrok domain, so `curl` reported that it could not resolve the host while Google's DNS resolved it. I switched the resolver to 8.8.8.8 and 1.1.1.1.
4. **An agent refused a probe on its own judgment.** In my first smoke test the agent declined to read `../.env` without calling the server, so nothing was logged. Re-asking with `../package.json` produced a real server-side rejection. In Session 3 all seven probes reached the server.
5. **Token hygiene.** I kept the token in `.env` and in VS Code's password prompt, rotated one token that was exposed during setup, and confirmed that no submission file contains it.
6. **The ngrok warning page.** Free ngrok shows an interstitial page to browsers. MCP clients are not affected, as the inspector showing `200` responses for `/mcp/qa` and `/mcp/dev` confirms.

9. Limitations and future work

- The sandbox is application-level, not an OS or container boundary. A small time-of-check/time-of-use window exists between path resolution and the file operation; agents have no tool that creates symlinks.
- Both roles share one bearer token, so QA is separated by endpoint, not by identity. Separate tokens per role are the next step.
- There is no rate limiting and no write locking. Two dev sessions editing the same file at once could overwrite each other; `if_version` locking would fix that.
- QA cannot write to the handoff files (see section 4).

- The Flutter code was never compiled or run. The agents had only file tools, so Sessions 1 and 2 verified it by inspection and `search_code`. The next step is `flutter pub get`, `flutter analyze` and tests in a Flutter environment.
- Session 3 found 0 issues, but I did not plant a defect, so it shows the audit workflow more than the audit's sensitivity. A follow-up would plant a flaw, such as an open Firestore rule, and check that QA catches it.
- In Session 3 the agent also listed two VS Code helper tools (`session_store_sql` and `multi_tool_use.parallel`). They come from the client, are not file tools, and I do not control them. The connector's QA endpoint itself exposes only the 4 tools.
- Stretch goals: bearer-token auth is done. `if_version` locking and a repo map are not.

10. Evidence index

Evidence	Location in `submission/`
Tool-call log	`logs/tool-calls.jsonl` , `log-summary.md` , `session1-summary.md` , `session2-summary.md` , `session3-summary.md`
Auth failures	`logs/auth-failures.jsonl`
First sandbox rejection (smoke test)	`logs-smoke-test/`
Handoff files	`handoff/`
Workspace git history	`workspace-git-log.txt` , screenshot 17
Test results	`test-output.txt` , screenshot 01
Session transcripts	`transcripts/session1-transcript.md` , `session2-transcript.md` , `session3-transcript.md`
Screenshots (listed in `screenshots/INDEX.txt`)	`screenshots/` (401 responses, sandbox errors, pm2, ngrok, tools picker)

The repository's `submission/` folder mirrors this evidence index, so every item above can be checked on GitHub: https://github.com/zekeyell/Agent-handoff-mcp/tree/main/submission